



Today's agenda

↳ Skyline Problem

↳ Min no. of refueling stops

↳ 1 more prob.



AlgoPrep



Q) The Skyline Problem

 Problem List < > ↺

Description Editorial Solutions Submissions

218. The Skyline Problem Solved ✓

Hard Topics Companies

A city's **skyline** is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return *the skyline formed by these buildings collectively*.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [lefti, righti, heighti]`:

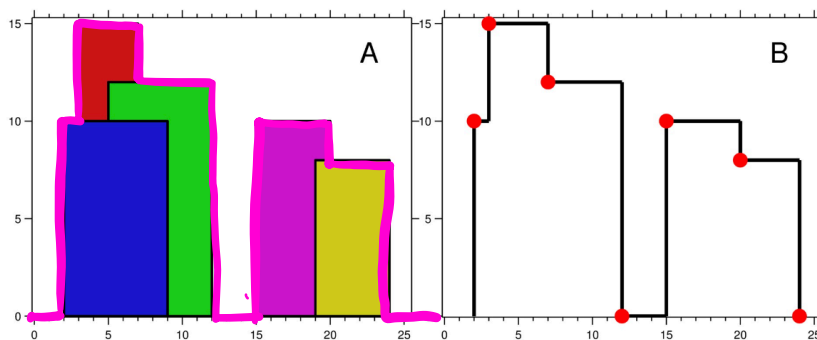
- `lefti` is the x coordinate of the left edge of the `ith` building.
- `righti` is the x coordinate of the right edge of the `ith` building.
- `heighti` is the height of the `ith` building.

You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height `0`.

The **skyline** should be represented as a list of "key points" **sorted by their x-coordinate** in the form `[[x1,y1],[x2,y2],...]`. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate `0` and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, `[...,[2 3],[4 5],[7 5],[11 5],[12 7],...]` is not acceptable; the three lines of height 5 should be merged into one in the final output as such: `[...,[2 3],[4 5],[12 7],...]`

Example 1:



Input: `buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]`

Output: `[[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]`

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

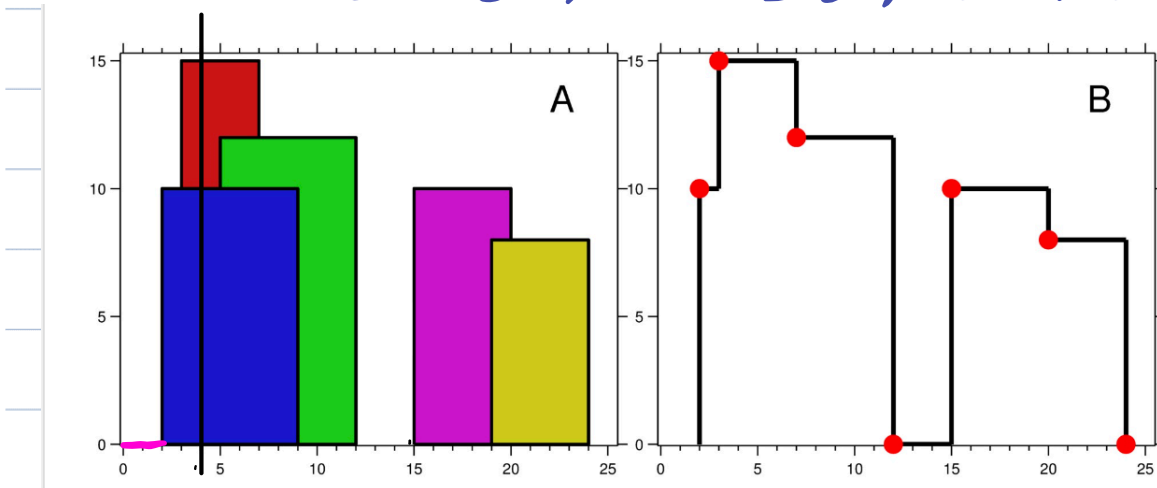
Input: `buildings = [[0,2,3],[2,5,3]]`

Output: `[[0,3],[5,0]]`



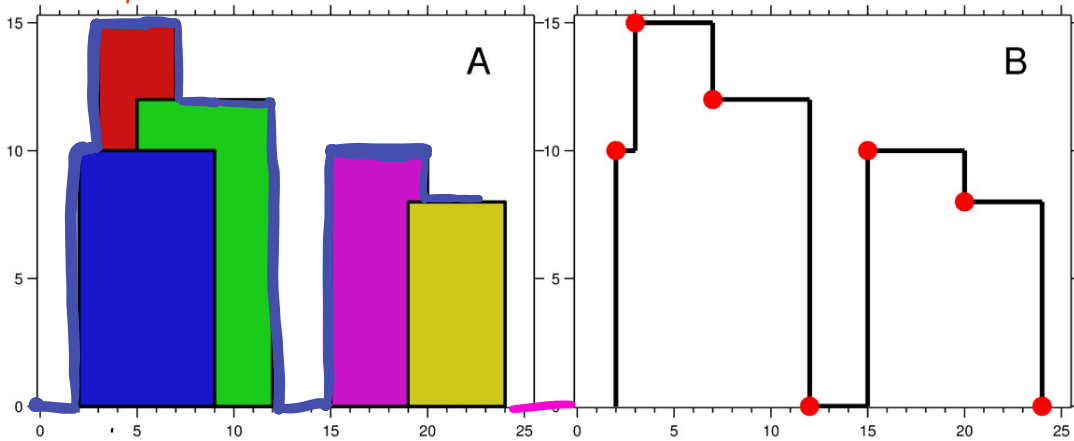
$[2, 9, 10]$ $[3, 7, 15]$ $[5, 12, 12]$ $[5, 20, 10]$ $[19, 24, 8]$

$[2, -10]$ $[9, 10]$ $[3, -15]$ $[7, 15]$ $[5, -12]$ $[12, 13]$ $[15, -10]$ $[20, 10]$ $[19, -8]$ $[24, 8]$



Obs1: The most relevant rectangle at any point will be the rectangle with max height.

Obs2: The Skyline Points can be at the start of a rectangle or at the end of the rectangle.



inc order

$[2, -10]$ $[3, -15]$ $[5, -12]$ $[7, 15]$ $[9, 10]$ $[12, 12]$ $[15, -10]$ $[19, -8]$ $[20, 10]$ $[24, 8]$

0 1 2 3 4
 $\{2, 10\}$ $\{3, 15\}$ $\{7, 12\}$ $\{12, 0\}$ $\{15, 10\}$
 5 6
 $\{20, 8\}$ $\{24, 0\}$



Result: ~~10~~ ~~15~~ ~~12~~ ~~10~~ ~~8~~ ~~0~~

Public class Pair implements Comparable<Pair>

int n;

int ht;

Public int compareTo (Pair o) {

if (this.n != o.n)

return this.n - o.n;

}

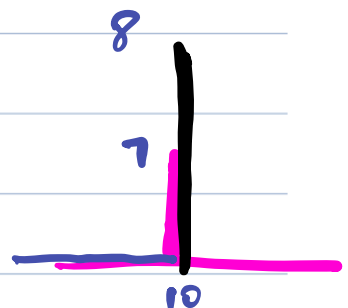
else {

return this.ht - o.ht;

}

}

}





11Pseudo code

```
List < List < Integer > > getSkyLine (int[][] buildings) {
```

```
    List < Pair > list = new ArrayList < > ();
```

```
    for (int i=0; i < buildings.length; i++) {
```

```
        int sp = buildings[i][0];
```

```
        int ep = buildings[i][1];
```

```
        int h = buildings[i][2];
```

```
        list.add (new Pair(sp, -h));
```

```
        list.add (new Pair(ep, h));
```

```
    }
```

```
    Collections.sort (list);
```

```
    List < List < Integer > > ans = new ArrayList < > ();
```

```
    PriorityQueue < Integer > PQ;
```

```
    int prevht = 0;
```

```
    PQ.add (0);
```

```
    for (int i=0; i < list.size(); i++) {
```

```
        Pair rem = list.get(i);
```

```
        if (rem.ht < 0) { PQ.add (-rem.ht); }
```

```
        else { PQ.remove (rem.ht); }
```

```
        if (prevht != PQ.peek()) {
```

```
            List < Integer > temp = new ArrayList < > ();
```

```
            temp.add (rem.n);
```

```
            temp.add (PQ.peek());
```

```
            ans.add (temp);
```

```
            prevht = PQ.peek();
```

```
    }
```

```
    }
```

```
}
```



Bolak till 9:35 PM



AlgoPrep